

Analysis of Algorithms

1. Introduction

An **algorithm** is a step-by-step procedure to solve a problem or perform a task.

Example

Algorithm to find the sum of two numbers:

1. Start
2. Read A and B
3. $\text{Sum} = A + B$
4. Print Sum
5. Stop

Algorithms should be:

- Correct
- Efficient
- Finite
- Easy to understand

2. Why Data Structures and Algorithms (DSA)?

Data Structure

A way of organizing and storing data efficiently.

Examples:

- Array
- Linked List
- Stack
- Queue
- Tree
- Graph

Why DSA?

1. Improves program performance.
2. Reduces execution time.
3. Optimizes memory usage.
4. Helps solve complex problems efficiently.
5. Important for coding interviews and placements.

Example

Searching an element in:

- Unsorted array → Linear Search → $O(n)$
- Sorted array → Binary Search → $O(\log n)$

Binary Search is much faster for large datasets.

3. Types of Data Structures

A. Primitive Data Structures

Basic data types provided by programming languages.

Examples:

- Integer
- Float
- Character
- Boolean

a = 10

b = 3.14

c = 'A'

d = True

B. Non-Primitive Data Structures

1. Linear Data Structures

Elements are arranged sequentially.

Examples:

- Array
- Linked List
- Stack
- Queue

10 → 20 → 30 → 40

2. Non-Linear Data Structures

Elements are arranged hierarchically.

Examples:

- Tree
- Graph

```
A
/ \
B  C
```

4. Asymptotic Notations

Used to measure algorithm efficiency.

A. Big O Notation - $O()$

Represents the **worst-case time complexity**.

Example

```
for i in range(n):
    print(i)
```

Operations = n

Time Complexity = **$O(n)$**

B. Omega Notation - $\Omega()$

Represents the **best-case complexity**.

Example

Linear Search:

arr = [10,20,30,40]

target = 10

Element found at first position.

Complexity = $\Omega(1)$

C. Theta Notation - $\Theta()$

Represents the **average or exact growth rate**.

Example

for i in range(n):

 print(i)

Best = Worst = n

Complexity = $\Theta(n)$

5. Time Complexity

Measures how execution time grows with input size.

Common Time Complexities

Complexity Name

$O(1)$ Constant

$O(\log n)$ Logarithmic

$O(n)$ Linear

$O(n \log n)$ Linearithmic

Complexity Name

$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential
$O(n!)$	Factorial

Examples

$O(1)$ - Constant

```
arr = [10,20,30]  
print(arr[0])
```

Only one operation.

$O(\log n)$ - Binary Search

```
while start <= end:  
    mid = (start + end) // 2
```

Search space reduces by half.

$O(n)$ - Linear

```
for i in range(n):  
    print(i)
```

Runs n times.

$O(n^2)$ - Nested Loop

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

Runs $n \times n$ times.

6. Space Complexity

Measures the memory used by an algorithm.

Formula

Space Complexity =
Input Space + Auxiliary Space

Example 1: $O(1)$

```
a = 10  
b = 20  
c = a + b
```

Uses fixed memory.

Space Complexity = $O(1)$

Example 2: $O(n)$

```
arr = []
```

```
for i in range(n):  
    arr.append(i)
```

Stores n elements.

Space Complexity = $O(n)$

Interview Questions

What is an Algorithm?

A finite sequence of steps used to solve a problem.

What is Time Complexity?

The amount of time an algorithm takes relative to input size.

What is Space Complexity?

The amount of memory required by an algorithm.

Difference between Time and Space Complexity?

Time Complexity	Space Complexity
Measures execution time	Measures memory usage
$O(n)$, $O(\log n)$ etc.	$O(n)$, $O(1)$ etc.

Which is better?

$O(1) > O(\log n) > O(n) > O(n \log n) > O(n^2) > O(2^n) > O(n!)$

(From most efficient to least efficient)

Frameworks for Describing and Analyzing Algorithms

A **framework** is a systematic way to describe an algorithm and measure its efficiency.

1. Describing an Algorithm

An algorithm is usually described using:

A. Natural Language

Example: Find the largest number in an array.

1. Start
2. Assume first element is largest
3. Compare with remaining elements
4. Update largest if needed
5. Print largest
6. Stop

B. Pseudocode

Algorithm Largest(A, n)

max = A[0]

for i = 1 to n-1

 if A[i] > max

 max = A[i]

return max

Pseudocode is language-independent.

C. Flowchart

Start

|

Read Array

|

Compare Elements

|

Largest Found?

|

Print Largest

|

Stop

2. Framework for Analyzing Algorithms

When analyzing an algorithm, we mainly focus on:

A. Input Size (n)

Input size determines the amount of work.

Example:

arr = [10,20,30,40,50]

Here,

$n = 5$

B. Time Complexity

Measures running time.

Example:

```
for i in range(n):  
    print(i)
```

Operations = n

Time Complexity = **$O(n)$**

C. Space Complexity

Measures memory used.

```
arr = [0] * n
```

Stores n elements.

Space Complexity = **$O(n)$**

3. Best, Average, and Worst Cases

Example: Linear Search

```
def search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

Best Case

Element found at first position.

Operations = 1

Complexity = $O(1)$

Average Case

Element found in middle.

Operations $\approx n/2$

Complexity = $O(n)$

Worst Case

Element at end or not present.

Operations = n

Complexity = $O(n)$

4. Asymptotic Analysis

Used to compare algorithms for large inputs.

Big O (Worst Case)

for i in range(n):

 print(i)

Complexity = **$O(n)$**

Omega (Best Case)

Linear Search first element found.

Complexity = **$\Omega(1)$**

Theta (Exact Growth)

for i in range(n):

 print(i)

Complexity = **$\Theta(n)$**

5. Example Analysis

Algorithm

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

Step 1

Outer loop = n

Step 2

Inner loop = n

Step 3

Total Operations

$$n \times n = n^2$$

Time Complexity

$$O(n^2)$$

Space Complexity

$$O(1)$$

Framework Checklist for Any Algorithm

Whenever you see a program, analyze it using this framework:

1. Identify Input Size

n = number of elements

2. Count Operations

Statements

Loops

Recursion calls

3. Determine Time Complexity

$O(1)$

$O(\log n)$

$O(n)$

$O(n \log n)$

$O(n^2)$

4. Determine Space Complexity

Extra variables $\rightarrow O(1)$

Extra array of size $n \rightarrow O(n)$

5. Consider Cases

Best Case

Average Case

Worst Case

Example: Linear Search

Problem

Find whether a target element exists in an array.

```
def linear_search(arr, target):
```

```
    for i in range(len(arr)):
```

```
        if arr[i] == target:
```

```
            return i
```

```
    return -1
```

Input

```
arr = [10, 20, 30, 40, 50]
```

```
target = 40
```

Step 1: Understand the Problem

We need to find the position of 40 in the array.

Array:

Index : 0 1 2 3 4

Value : 10 20 30 40 50

Step 2: Identify Input Size

Input size is represented by **n**.

n = number of elements in array

Here:

n = 5

Step 3: Trace the Algorithm

Iteration 1

arr[0] = 10

10 == 40 ? No

Operations = 1 comparison

Iteration 2

arr[1] = 20

20 == 40 ? No

Operations = 2 comparisons

Iteration 3

arr[2] = 30

30 == 40 ? No

Operations = 3 comparisons

Iteration 4

arr[3] = 40

40 == 40 ? Yes

Return index 3

Operations = 4 comparisons

Step 4: Count Basic Operations

The important operation is:

`arr[i] == target`

This comparison happens repeatedly.

Worst Case

Suppose target is not present.

`arr = [10,20,30,40,50]`

`target = 100`

Comparisons:

`10 == 100`

`20 == 100`

`30 == 100`

`40 == 100`

`50 == 100`

Total comparisons = n

Step 5: Calculate Time Complexity

Best Case

Target found at first position.

`10 == 10`

Comparisons = 1

Time Complexity = $O(1)$

Average Case

Target found in middle.

Comparisons $\approx n/2$

Time Complexity = $O(n)$

Worst Case

Target at last position or absent.

Comparisons = n

Time Complexity = $O(n)$

Step 6: Calculate Space Complexity

Variables used:

i

target

Only a few extra variables.

No extra array created.

Space Complexity = $O(1)$

(Constant space)

Step 7: Asymptotic Notations

Big O (Worst Case)

$O(n)$

Need to check all elements.

Omega (Best Case)

$\Omega(1)$

Found immediately.

Theta (Average Case)

$\Theta(n)$

Usually scans a significant part of the array.

Step 8: Final Analysis Table

Property	Result
Problem	Search an element
Input Size	n
Best Case	$O(1)$
Average Case	$O(n)$
Worst Case	$O(n)$
Space Complexity	$O(1)$
Big O	$O(n)$
Omega	$\Omega(1)$
Theta	$\Theta(n)$

Using Asymptotic Notation for Running-Time Analysis

Asymptotic Notation is used to describe how the running time of an algorithm grows as the input size (**n**) becomes very large.

Instead of counting exact seconds, we focus on the **growth rate**.

Types of Asymptotic Notations

1. Big O Notation - $O()$

Represents the **Worst Case**.

Example

```
for i in range(n):  
    print(i)
```

Operations:

n

Running Time:

$O(n)$

2. Omega Notation - $\Omega()$

Represents the **Best Case**.

Example: Linear Search

```
if arr[0] == target:  
    return 0
```

Found immediately.

$\Omega(1)$

3. Theta Notation - $\Theta()$

Represents the **Exact/Average Growth Rate**.

Example

```
for i in range(n):  
    print(i)
```

Best and Worst are both n.

$\Theta(n)$

Example 1: Constant Time

a = 10

b = 20

c = a + b

Operations = 1

$O(1)$

$\Omega(1)$

$\Theta(1)$

Example 2: Linear Time

```
for i in range(n):  
    print(i)
```

Operations = n

$O(n)$

$\Omega(n)$

$\Theta(n)$

Example 3: Quadratic Time

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

Operations:

$n \times n = n^2$

$O(n^2)$

$\Omega(n^2)$

$\Theta(n^2)$

Example 4: Logarithmic Time

$i = 1$

```
while i < n:  
    i *= 2
```

Growth:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \dots$

Operations:

$\log n$

$O(\log n)$

$\Omega(\log n)$

$\Theta(\log n)$

Example 5: $n \log n$ Time

for i in range(n):

$j = 1$

 while $j < n$:

$j *= 2$

Operations:

$n \times \log n$

$O(n \log n)$

$\Omega(n \log n)$

$\Theta(n \log n)$

Rules for Writing Asymptotic Notation

Rule 1: Remove Constants

$5n + 10$

becomes

$O(n)$

Rule 2: Keep Highest Power

$n^2 + n + 10$

becomes

$O(n^2)$

Rule 3: Ignore Lower Terms

$$n^3 + n^2 + n$$

becomes

$$O(n^3)$$

Quick Examples

Running Time Asymptotic Notation

$$7 \quad O(1)$$

$$3n + 5 \quad O(n)$$

$$n^2 + 10n \quad O(n^2)$$

$$n^3 + n^2 + n \quad O(n^3)$$

$$\log_2 n + 5 \quad O(\log n)$$

$$n \log n + n \quad O(n \log n)$$

Example :

Convert the following to Big O:

$$T(n) = 4n^2 + 3n + 20$$

Step 1:

Remove constants

Step 2:

Keep highest power

Answer:

$$O(n^2)$$

Most Important Complexities

$O(1)$ Constant

$O(\log n)$ Logarithmic

$O(n)$ Linear

$O(n \log n)$

$O(n^2)$ Quadratic

$O(n^3)$ Cubic

$O(2^n)$ Exponential

$O(n!)$ Factorial

Efficiency order:

$O(1) < O(\log n) < O(n) < O(n \log n)$

$< O(n^2) < O(n^3) < O(2^n) < O(n!)$

(Left is better, right is worse.)

Finding Time Complexity of Iterative and Recursive Algorithms

1. Iterative Algorithms

Example 1: Single Loop

```
for i in range(n):  
    print(i)
```

Analysis

- Loop runs n times.
- Basic operation = `print()`

Operations = n

Time Complexity = $O(n)$

Example 2: Nested Loops

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

Analysis

- Outer loop = n
- Inner loop = n

Operations = $n \times n = n^2$
Time Complexity = $O(n^2)$

Example 3: Logarithmic Loop

```
i = 1
```

```
while i < n:  
    i = i * 2
```

Analysis

Values of i :

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \dots$

Number of iterations:

$\log_2 n$

Time Complexity = $O(\log n)$

Example 4: $n \log n$

```
for i in range(n):  
    j = 1
```

```
    while j < n:  
        j *= 2
```

Analysis

- Outer loop = n

- Inner loop = $\log n$

Operations = $n \times \log n$

Time Complexity = $O(n \log n)$

2. Recursive Algorithms

A recursive function calls itself.

Example 1: Factorial

```
def fact(n):  
    if n == 0:  
        return 1  
  
    return n * fact(n-1)
```

Recursive Calls

```
fact(5)  
fact(4)  
fact(3)  
fact(2)  
fact(1)  
fact(0)
```

Number of calls:

n

Time Complexity = $O(n)$

Example 2: Sum of First n Numbers

```
def sum_n(n):  
    if n == 0:  
        return 0
```

```
return n + sum_n(n-1)
```

Analysis

$$T(n) = T(n-1) + 1$$

Number of calls:

n

Time Complexity = $O(n)$

Example 3: Binary Search (Recursive)

```
def binary_search(arr, low, high, target):
```

```
    if low > high:
```

```
        return -1
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == target:
```

```
        return mid
```

```
    elif arr[mid] < target:
```

```
        return binary_search(arr, mid+1, high, target)
```

```
    else:
```

```
        return binary_search(arr, low, mid-1, target)
```

Analysis

Each call halves the search space.

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8$$

Number of calls:

$\log n$

Time Complexity = $O(\log n)$

Example 4: Fibonacci (Recursive)

```
def fib(n):  
    if n <= 1:  
        return n  
  
    return fib(n-1) + fib(n-2)
```

Recursive Tree

```
fib(5)  
├── fib(4)  
│   ├── fib(3)  
│   │   ├── fib(2)  
│   │   └── fib(3)  
│   │       ├── fib(2)  
│   │       └── fib(1)
```

Many calls are repeated.

Time Complexity = $O(2^n)$

Quick Method to Find Complexity

For Iterative Algorithms

Pattern	Complexity
One loop	$O(n)$
Two nested loops	$O(n^2)$
Three nested loops	$O(n^3)$
Divide/Multiply by 2	$O(\log n)$
Loop + log loop	$O(n \log n)$

For Recursive Algorithms

Recurrence	Complexity
$T(n)=T(n-1)+1$	$O(n)$
$T(n)=T(n/2)+1$	$O(\log n)$
$T(n)=2T(n-1)+1$	$O(2^n)$
$T(n)=T(n-1)+T(n-2)$	$O(2^n)$

Practice Questions

Find the time complexity:

Q1

```
for i in range(n):  
    print(i)
```

Answer: **$O(n)$**

Q2

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```

Answer: **$O(n^2)$**

Q3

```
def fun(n):  
    if n == 0:  
        return  
  
    fun(n-1)
```

Answer: **$O(n)$**

Q4

```
def fun(n):
```

```
    if n <= 1:
```

```
        return
```

```
    fun(n-1)
```

```
    fun(n-1)
```

Answer: **$O(2^n)$**